

9 anos de ensino de Redes com Python

onde dá conta de tudo, e onde foi preciso buscar alternativas

Prof. Paulo Matias (UFSCar)

sábado, 6 de junho, 14:50

Caipyra 2026 – Palestras – Auditório Fernão

Iniciante

Resumo da palestra

- ▶ o recorte de 9 anos é Redes de Computadores com Python desde 2018
- ▶ a pilha implementada passa por IRC, TCP, IPv4, SLIP e hardware real em FPGA
- ▶ desde 2022, leciono Tecnologia de Comunicação, que é disciplina correlata
- ▶ nesse contraste, Python funciona bem em simulação e processamento offline
- ▶ no modem V.21, tempo real levou à migração para C++ e Rust

A pergunta da palestra é menos “Python serve?” e mais “em que papel Python serve melhor?”.

Ensino de Redes

Python funciona bem quando a disciplina quer que estudantes implementem protocolos, não apenas configurem equipamentos ou o sistema operacional.

A aposta não foi “Python é mais fácil”. A aposta foi: a linguagem deixa a pilha de protocolos suficientemente visível para testar, depurar e integrar em ambiente real dentro de um semestre.

O contraste que motivou a disciplina

Ênfase comum

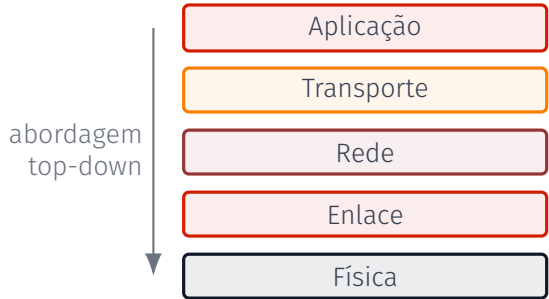
- ▶ configurar roteadores, switches e serviços
- ▶ usar ferramentas do sistema operacional
- ▶ diagnosticar redes já prontas
- ▶ entender protocolos pelo comportamento externo

Ênfase na UFSCar

- ▶ implementar camadas da pilha
- ▶ construir contratos entre camadas
- ▶ passar de testes locais para cenário físico
- ▶ entender protocolos pelo estado interno

O que se aprende em Redes?

- ▶ modelo em camadas
- ▶ encapsulamento
- ▶ multiplexação
- ▶ controle de erro
- ▶ temporização
- ▶ roteamento
- ▶ interoperabilidade



Narrativa das práticas



Cada prática remove uma camada do Linux e substitui por uma implementação dos estudantes em Python. No final, a pilha roda sobre portas seriais reais.

Camada de aplicação: IRC

Contrato externo

O servidor é testado como servidor de verdade: aceita conexões, recebe comandos IRC e envia linhas de resposta.

```
def trata_comando(conexao, cmd):
    cmd, args = (cmd.split(b" ", 1) + [b""])[1:]
    cmd = cmd.upper()

    if cmd == b"NICK":
        by_nickname[args.lower()] = conexao
        conexao.nick = args
    elif cmd == b"PING":
        conexao.enviar(b":server PONG server :%s\r\n" % args)
    elif cmd == b"PRIVMSG":
        dest = args.split(b" ", 1)[0].lower()
        contents = args.split(b" :", 1)[1]
        by_nickname[dest].enviar(
            b":%s PRIVMSG %s :%s\r\n" % (conexao.nick, dest, contents))
```

Por que Python ajuda na aplicação?

O aluno precisa modelar

- ▶ usuários por apelido
- ▶ canais como conjuntos
- ▶ comandos textuais
- ▶ broadcast seletivo
- ▶ fragmentação de entrada

A linguagem não vira o assunto

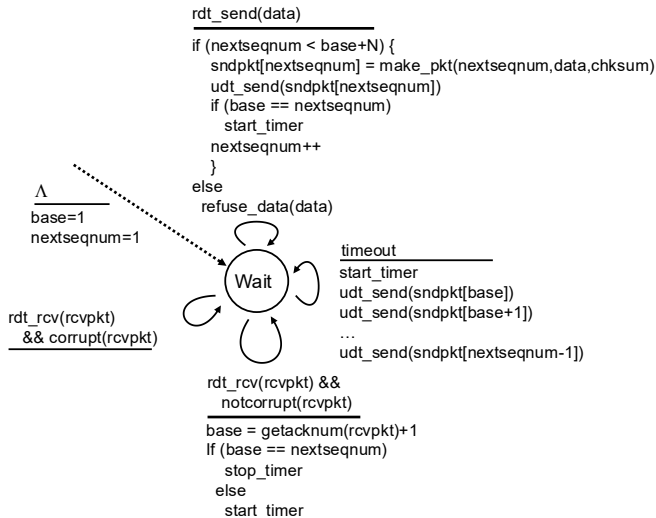
- ▶ dict, set, bytes
- ▶ callbacks simples
- ▶ testes por socket real
- ▶ pouca cerimônia para I/O
- ▶ bugs visíveis no protocolo

Camada de transporte: TCP

P2: a prática mais difícil

- ▶ TCP é uma máquina de estados
- ▶ eventos mudam variáveis como `base`, janela e timer
- ▶ Go-Back-N é o modelo didático mais próximo do transmissor

A ideia central: confiabilidade aparece de ACKs, temporizadores e retransmissões, não de uma chamada mágica de socket.



TCP em Python: eventos viram callbacks

Chegada de segmento da camada de rede

```
class Servidor:
    def __init__(self, rede, porta):
        self.rede = rede
        self.rede.registrar_recebedor(
            self._rdt_rcv)

    def _rdt_rcv(self, src, dst, segment):
        ...
        conexao._rdt_rcv(seq_no, ack_no,
                          flags, payload)
```

Como o asyncio ajuda

- ▶ a camada IP chama `_rdt_rcv` quando chega um segmento
- ▶ esse método é a transição `rdt_rcv(rcvpkt)` da FSM
- ▶ o código não precisa ficar preso em um `while` esperando pacote

TCP em Python: timer e ACK atualizam estado

Expiração de timer

```
def _start_timer(self):
    timeout = self.estimated_rtt + \
              4*self.dev_rtt
    loop = asyncio.get_event_loop()
    self.timer = loop.call_later(
        timeout,
        self._timer_callback)

def _timer_callback(self):
    payload = self.not_acked_queue[:MSS]
    if payload:
        segmento = ... # retransmissão
        self.servidor.rede.enviar(
            segmento, self.dst_addr)
    self._start_timer()
```

ACK cumulativo recebido

```
if ack_no > self.send_base:
    delta = ack_no - self.send_base
    self.not_acked_queue = \
        self.not_acked_queue[delta:]
    self.send_base = ack_no

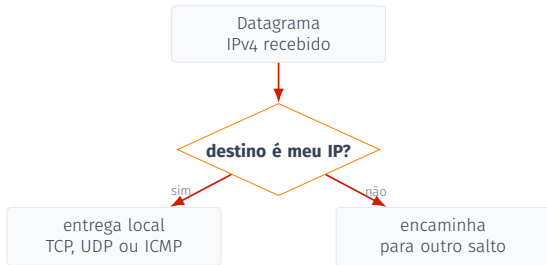
if self.not_acked_queue:
    self._start_timer()
else:
    self._stop_timer()

self._send_win()
```

Com `asyncio`, eventos da FSM aparecem como chamadas de função: chegada de ACK e expiração de timer atualizam filas, janela e temporizador.

IPv4: destino final ou roteador?

A primeira decisão



Por que isso importa

- ▶ o mesmo código pode receber pacotes destinados à própria máquina
- ▶ se o destino é local, o payload sobe para a camada de transporte
- ▶ se não é local, o sistema age como roteador
- ▶ só nesse segundo caso entra a tabela de rotas

O slide seguinte mostra justamente essa parte de roteador: escolher o próximo salto por *longest prefix match*.

Camada de rede: IPv4

Escolher o próximo salto

```
def _next_hop(self, dest_addr):
    dest_addr = str2naddr(dest_addr)
    best_match, best_match_n = None, -1
    for cidr, next_hop in self.tabela:
        net, nbits = cidr.split("/")
        net = str2naddr(net)
        nbits = int(nbits)
        mask = 0xffffffff >> (32-nbits)
        mask = mask << (32-nbits)
        is_match = net == (dest_addr & mask)
        if nbits > best_match_n and is_match:
            best_match = next_hop
            best_match_n = nbits
    return best_match
```

Longest prefix match

- ▶ roteador consulta uma tabela: prefixo → próximo salto
- ▶ 192.168.10.0/24 cobre todos os destinos com esses 24 bits iniciais
- ▶ *longest prefix match*: entre rotas compatíveis, vence a mais específica
- ▶ isso permite uma rota geral e exceções mais detalhadas na mesma tabela

No código, a máscara compara só o prefixo; **best_match_n** guarda a rota compatível mais específica.

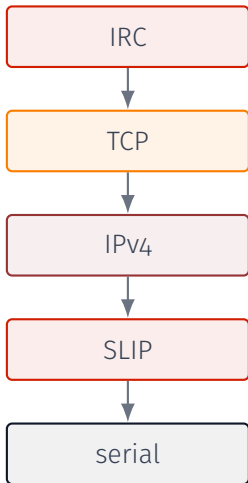
Camada de enlace: SLIP

Transformar fluxo de bytes em quadros

```
def enviar(self, datagrama):
    datagrama = datagrama.replace(b"\xdb", b"\xdb\xdd")
    datagrama = datagrama.replace(b"\xc0", b"\xdb\xdc")
    self.linha_serial.enviar(b"\xc0" + datagrama + b"\xc0")

def __raw_recv(self, dados):
    for c in dados:
        if c == 0o300 and self.buf:
            self.callback(self.buf)
            self.buf = b""
        elif c == 0o333:
            self.inside_escape = True
        else:
            self.buf += bytes([c])
```

O padrão que se repete



Interface de cada camada

`registrar_recebedor(callback)`

`enviar(payload, destino)`

O callback explicita a direção “subindo”, e o método `enviar` explicita a direção “descendo”.

Testes automatizados como parte do desenho

Durante o semestre

- ▶ alunos trabalham em casa ou em sala
- ▶ cada passo tem testes independentes
- ▶ feedback rápido antes do teste físico
- ▶ liberdade para reorganizar a implementação

No final

- ▶ código exercitado em rede real
- ▶ testes cobrem o caminho crítico
- ▶ a maioria funciona de primeira
- ▶ erros restantes aparecem como comportamento de rede

Infraestrutura de avaliação

2018–2020



Autolab

2021–2025.1



GitHub CI +
Apps Script

2025.2–2026.1



GitHub Class-
room + unittest

2026.2?



Classroom
descontinuado

A tecnologia de avaliação mudou, mas o contrato didático permaneceu:
entregar código executável, testável e integrável.

2018/2: linguagem livre, adoção de Python

Ativ.	Python	C/BSV	Java	Lua	Total
P1	22	1	1	1	25
P2	19	0	0	0	19
P3	22	0	0	0	22
P4	15	0	0	0	15
P5	18	0	0	0	18

Contrato da prática

- ▶ todas as práticas aceitavam outra linguagem
- ▶ na P1, testes sobem `./servidor` e falam por socket
- ▶ da P2 em diante, testes importam módulos Python

Custo prático

- ▶ outra linguagem exigia teste manual ou portar a suíte
- ▶ em 2018/2, P2–P5 tiveram só entregas em Python
- ▶ depois disso, as entregas seguem em Python até 2026/1

Python antes de Redes mudou de lugar

2018

Quase ausente antes de Redes.

2019–2021

Exposição por trajetórias não típicas: Matemática Computacional, Teoria dos Grafos, PDI, IA, Tópicos.

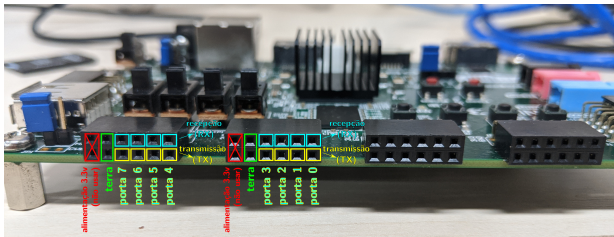
2022–2026

Fluxo anterior: AED2 em 2022/1; depois AED1 em 2023/2; CAP e Probabilidade em 2024/2.



O mesmo indicador muda de significado: primeiro trajetória individual; depois estrutura curricular observada nas turmas de Redes.

Integração com camada física



Zybo Z7-20

- ▶ FPGA expõe 8 portas seriais
- ▶ RX/TX/GND conectados com fios
- ▶ filas de TX/RX em hardware
- ▶ estudantes rodam a pilha Python nas placas

Driver 100% Python, sem módulo C

UIO + mmap + descritor de arquivo

```
import os, mmap, fcntl, struct, asyncio

class ZyboSerialDriver:
    def __init__(self, device="/dev/uio/user_io"):
        self.fd = os.open(device, os.O_RDWR)
        fcntl.fcntl(self.fd, fcntl.F_SETFL, os.O_NONBLOCK)
        self.mm = mmap.mmap(self.fd, 0x1000)
        loop = asyncio.get_event_loop()
        loop.add_reader(self.fd, self.__irq_handler)
        self.__irq_unmask()

    def enviar(self, port, data):
        for b in data:
            offset = port * 4
            self.mm[offset:offset+4] = struct.pack("I", b)

    def __irq_handler(self):
        os.read(self.fd, 4) # reconhece a IRQ no UIO
        elem, = struct.unpack("i", self.mm[0:4])
        port, b = elem >> 8, elem & 0xff
```

- ▶ /dev/uio/user_io
- ▶ mmap dos registradores
- ▶ IRQ via descritor de arquivo
- ▶ PTY via os.openpty()
- ▶ sem extensão Python
- ▶ sem módulo C

O kernel fornece as interfaces; o FPGA faz a temporização.

Stanford CS144: parentesco real, diferenças importantes

Eixo	Stanford CS144 / Minnow	UFSCar Redes / Python
Entrada	Check 0-1: <code>webget</code> , <code>ByteStream</code> , <code>Reassembler</code> ; prepara TCP por abstrações de fluxo.	P1: servidor IRC real; a aplicação vem primeiro para dar destino à pilha.
TCP	<code>TCPReceiver</code> + <code>TCPsender</code> : seq. 32-bit, janela, retransmissão; interopera via TUN com TCP do Linux.	P2: ponta servidor sobre raw socket; handshake, ACK, FIN, timeout/RTT, AIMD; depois conversa com IRC.
L3/L2	Check 5: Ethernet + ARP + TAP. Check 6: roteador IP que ignora TCP/ARP/Ethernet. Check 7: Internet virtual sobre UDP.	P3: IPv4 host/roteador com LPM, TTL e ICMP Time Exceeded. P4: SLIP. S1: serial real em FPGA.
Ambiente	C++ moderno: <code>Rail</code> , <code>CMake</code> , sanitizers, <code>tidy/format</code> , testes e benchmark de desempenho.	Python: <code>bytes</code> , <code>struct</code> , <code>asyncio</code> , raw sockets, <code>mmap</code> ; testes importam as camadas.
Leitura	Pilha como biblioteca de sistemas, cuidadosamente fatorada em objetos.	Substituição incremental das camadas do Linux até uma rede física.

Quando Python deixa de ser a melhor escolha?

Tecnologia de Comunicação

Prática	Linguagem	Motivo principal
Modem V.21	C++ ou Rust	processamento em tempo real de áudio
Ping E1/HDLC/ICMP	Bluespec	circuito síncrono em hardware
Ethernet 10BASE-T/ARP	Bluespec	CSMA/CD completo em hardware
Antenas	Python	configuração de PyNEC/OpenEMS
802.11a/g offline	Python	processamento I/Q gravado

Modem V.21: a quebra do estilo

2022



Python: gabarito ok,
alunos com underrun

2023



C++: áudio ok,
bugs de memória

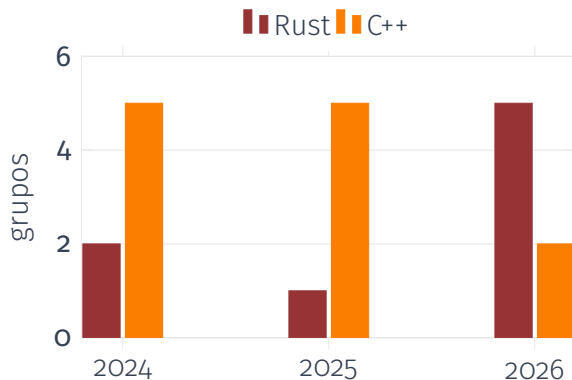
2024-2026



C++ ou Rust

Aqui a taxa de processamento passa a ser parte da atividade. O risco deixa de ser só “o protocolo está errado” e vira “o código correto não acompanha o tempo real”.

C++ ou Rust no modem



Leitura cautelosa

- ▶ Rust cresceu em 2026
- ▶ exposição prévia formal era baixa
- ▶ LLMs podem reduzir o custo percebido
- ▶ o dado não prova causalidade

Por que Rust, se quase ninguém viu Rust?

No recorte curricular

- ▶ BCC/EnC São Carlos desde 2018: o planos com Rust
- ▶ fora do recorte: Sorocaba chegou antes, em 2023/2, com Tópicos Avançados em Linguagens de Programação
- ▶ sem pré-requisito formal (como Python em Redes em 2018)

Motivo pragmático

- ▶ menos corrupção sutil de memória
- ▶ erros aparecem cedo no compilador
- ▶ starter code pode controlar a complexidade
- ▶ demanda de mercado ajuda a justificar

Por que não ficar sempre no ecossistema Python?

Alternativas possíveis

- ▶ Numba para kernels numéricos
- ▶ Cython para extensões compiladas
- ▶ MyHDL e Amaranth para gerar hardware
- ▶ PyPy para acelerar Python puro

Critério didático

- ▶ starter code precisa ser pequeno
- ▶ erros precisam ser claros para iniciantes
- ▶ integração não pode virar o centro da prática
- ▶ às vezes uma linguagem dedicada tem ecossistema mais limpo

Por que Bluespec em E1 e Ethernet?

Filosofia

- ▶ capturar erros em tempo de compilação
- ▶ linguagem cuja semântica representa circuitos síncronos
- ▶ alto nível sem esconder o hardware

Comparação com Python

- ▶ MyHDL/Amaranth são ótimos geradores
- ▶ mas funcionam como bibliotecas Python
- ▶ para iniciantes, a semântica da linguagem importa

Onde Python continua perfeito em Telecom

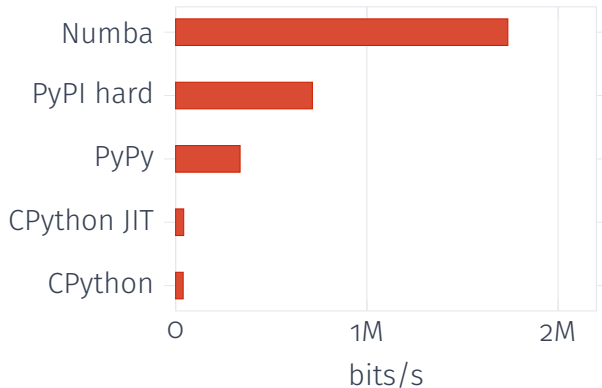
Simulação de antenas

- ▶ Python configura PyNEC/OpenEMS
- ▶ bibliotecas numéricas fazem o trabalho pesado
- ▶ ótimo papel de linguagem de orquestração

Wi-Fi 802.11a/g offline

- ▶ transceptor com NumPy/SciPy
- ▶ gravação I/Q de SDR
- ▶ processamento fora do tempo real
- ▶ testes precisam rodar em tempo razoável
- ▶ gargalo atual: Viterbi

Viterbi: benchmark do gargalo



Interpretação

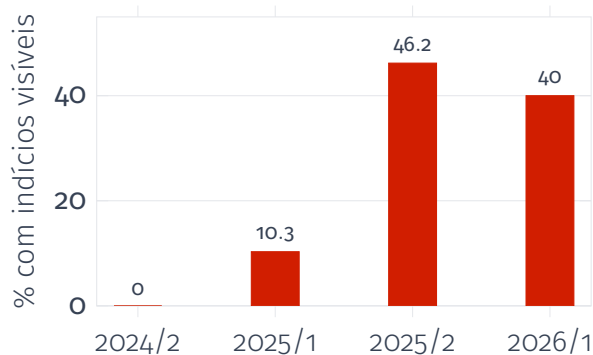
- ▶ JIT built-in: ganho pequeno
- ▶ PyPy acelera o kernel puro
- ▶ PyPI viterbi é hard-decision
- ▶ Numba é rápido, mas muda a dificuldade

JIT experimental do Python 3.14

Distro	Pacote	JIT	comentário
Arch Linux	python	×	JIT exige clang 19; Arch usa clang 22.1.6, e clang19 está só no AUR
Debian trixie	python3	×	Python 3.13; sem <code>sys._jit</code>
Debian forky	python3.14	✓	testing
Fedora 44	python	✓	disponível
Ubuntu 26.04 LTS	python3	✓	disponível
Nix unstable	python314	×	nixpkgs #331764
OpenSUSE Tumbleweed	python314	✓	disponível

Para a prática, o JIT é uma curiosidade operacional, não uma estratégia de desempenho: no benchmark isolado, o ganho foi cerca de 4%.

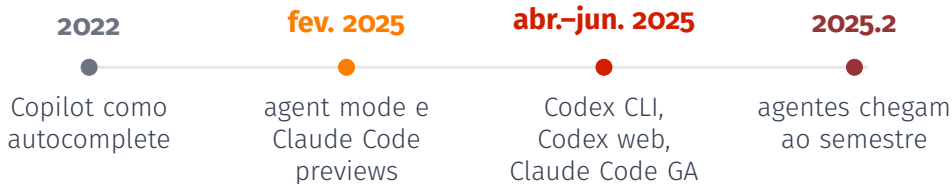
LLMs nas práticas de Redes



Cautela metodológica

- ▶ uso de LLM era permitido
- ▶ não é autoria comprovada
- ▶ são vestígios visíveis em repositórios
- ▶ salto claro em 2025/2

O que mudou em 2025



A ruptura não é “IA escreve código”. A ruptura é: agentes leem o repositório, editam patches, rodam testes e iteram com o ambiente da prática.

Telecom: P1/Rust e Bluespec são leituras separadas

telecom-p1: modem V.21

Período	repos	indícios de LLM
2024/1	7	0
2025/1	6	2 fortes, ambos em C++
2026/1	8	2 fracos, ambos em Rust

- ▶ P1 é a única prática de Telecom com opção Rust
- ▶ não há causalidade comprovada entre LLMs e aumento de Rust

Bluespec: percepção pessoal

- ▶ sem medida formal
- ▶ GPT-5.4 mais errava que acertava em projetos complexos
- ▶ GPT-5.5 mostra evolução clara
- ▶ por enquanto, ainda exige bastante interação humana

O problema filosófico

Em disciplinas de final de curso, o mercado espera que estudantes saibam operar agentes LLM.

Mas se um agente consegue resolver a prática incremental quase inteira sem direção humana, a atividade ainda está medindo aprendizagem de Redes ou só execução de testes?

Possíveis respostas didáticas

- ▶ manter testes automatizados, mas exigir defesa de decisões de protocolo
- ▶ avaliar depuração guiada, não só entrega que passa
- ▶ pedir comparação entre alternativas de implementação
- ▶ tornar explícito o uso de agentes e seus limites
- ▶ preservar práticas em que o mundo físico revela erros de integração

Fechamento

Python em Redes bom ajuste para implementar protocolos testáveis.

C++, Rust e Bluespec em Telecom tempo real e hardware mudam o critério.

Agentes LLM desde 2025 avaliação precisa medir decisão, explicação e depuração.

A pergunta central continua a mesma: que ferramenta deixa o fenômeno técnico mais visível para o aluno no tempo que a disciplina tem?

Materiais das disciplinas

Redes de Computadores

`redes.matias.co.in`

Tecnologia de Comunicação

`telecom.matias.co.in`